

Q1 · Arrays

Second Largest Without Duplicates

Given an integer array, find and return the second largest unique element. If no such element exists, return -1.

Input: [12, 35, 1, 10, 34, 1]

Output: 34

Input: [10, 10, 10]

Output: -1

Constraint: Do not use sorting. $O(n)$ solution expected.

A1:

```
int secondLargest(vector<int> &arr)
{
    int largest = arr[0];
    int slargest = -1;

    for(int i = 1; i < arr.size(); i++)
    {
        if(arr[i] > largest)
        {
            slargest = largest;
            largest = arr[i];
        }
        else if(arr[i] > slargest && arr[i] != largest)
        {
            slargest = arr[i];
        }
    }

    return slargest;
}
```

Q2 · Arrays

Rotate Array K Steps to the Right

Given an array of integers and a number k , rotate the array to the right by k positions in-place.

Input: [1, 2, 3, 4, 5, 6, 7], $k = 3$

Output: [5, 6, 7, 1, 2, 3, 4]

Input: [-1, -100, 3, 99], $k = 2$

Output: [3, 99, -1, -100]

Handle edge case: k may be larger than array length.

A2:

```
void rotateArray(vector<int>& arr, int k){
    int n = arr.size();
    k = k % n;
    reverse(arr.begin(), arr.end());
    reverse(arr.begin(), arr.begin() + k);
    reverse(arr.begin() + k, arr.end());
}
```

Valid Anagram Check

Write a function that takes two strings and returns `true` if one is an anagram of the other, `false` otherwise. Comparison should be case-insensitive and ignore spaces.

Input: "Listen", "Silent"

Output: true

Input: "hello world", "world hello"

Output: true

Input: "rat", "car"

Output: false

Do not use any built-in sort function. Use a frequency map.

A3:

```
bool isAnagram(string s1, string s2){
    unordered_map<char, int> freq;

    for(int i = 0; i < s1.length(); i++){
        if(s1[i] != ' ') {
            freq[tolower(s1[i])]++;
        }
    }

    for(int i = 0; i < s2.length(); i++){
        if(s2[i] != ' ') {
            freq[tolower(s2[i])]--;
        }
    }

    for(auto it = freq.begin(); it != freq.end(); it++){
        if(it->second != 0){
            return false;
        }
    }
    return true;
}
```

Longest Substring Without Repeating Characters

Given a string, find the length of the longest substring that contains no repeating characters.

Input: "abcabcbb"

Output: 3 (substring: "abc")

Input: "bbbbbb"

Output: 1 (substring: "b")

Input: "pwwkew"

Output: 3 (substring: "wke")

A4:- Sliding window

```
int lengthOfLongestSubstring(string s){
    unordered_set<char> st;

    int left = 0;
    int maxLen = 0;

    for(int right = 0; right < s.length(); right++){
        while(st.find(s[right]) != st.end()) {
            st.erase(s[left]);
            left++;
        }
        st.insert(s[right]);
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Q8 · API Integration

Weather Summary with Conditional Logic

Using the Open-Meteo API (free, no API key required), fetch the current temperature and weather code for any city of your choice. Based on the weather code returned, print a human-readable weather condition.

Base URL:

<https://api.open-meteo.com/v1/forecast>

Required query parameters:

latitude = {your chosen city's latitude}
longitude = {your chosen city's longitude}
current_weather = true

Sample response (current_weather object):

```
{
  "temperature": 18.2,
  "windspeed": 12.4,
  "weathercode": 3
}
```

Weather code reference (handle at least 5):

0 → Clear sky
1 → Mainly clear
2 → Partly cloudy
3 → Overcast
45 → Fog
61 → Light rain
80 → Rain showers
95 → Thunderstorm

Expected console output:

City : London
Temperature: 18.2°C
Condition : Overcast

A8-

```
import requests
```

```
weather_codes = {  
    0: "Clear sky",  
    1: "Mainly clear",  
    2: "Partly cloudy",  
    3: "Overcast",  
    45: "Fog",  
    61: "Light rain",  
    80: "Rain showers",  
    95: "Thunderstorm"  
}
```

```
params = {  
    "latitude": 51.5074,  
    "longitude": -0.1278,  
    "current_weather": "true"  
}
```

```
try:  
    response = requests.get(  
        "https://api.open-meteo.com/v1/forecast",  
        params=params,  
        timeout=10  
    )  
  
    response.raise_for_status()  
  
    data = response.json()  
  
    weather = data["current_weather"]  
  
    temperature = weather["temperature"]  
  
    code = weather["weathercode"]  
  
    print("City      : London")  
    print(f"Temperature: {temperature}°C")  
    print(  
        f"Condition : {weather_codes.get(code, 'Unknown')}"  
    )
```

```
except requests.exceptions.RequestException as e:  
    print(f"Request failed: {e}")
```